# EXPERIMENT – 9

**NAME: VEDHASREE S**

**REGISTER NUMBER: 240701580**

**Design input forms that validate data (e.g., email, phone number) and display error messages.**

To design a user input form that validates data such as name, email, and phone number using HTML, CSS, JavaScript, and Validator.js, and displays appropriate error messages for invalid inputs.

## EXPLANATION

A web form is used to collect user input in web applications. However, user inputs may often be incorrect or incomplete. To handle this, form validation is used. Form validation ensures that the data entered by the user follows correct format and rules before submission.

In this experiment:

- HTML is used to create the structure of the form.
- CSS is used to style the form and improve user interface.
- JavaScript is used to perform validation logic.
- Validator.js is a JavaScript library used to simplify validation of common fields such as email and empty values.

The form validates:

- Name → Only alphabets allowed
- Email → Must follow valid email format
- Phone number → Must be 10 digits and start with 6–9

Error messages are displayed below each input field to guide the user.

## PROCEDURE

- Create an HTML form with input fields (name, email, phone).

- Style the form using CSS for better UI.
- Include Validator.js library for validation support.
- Write JavaScript to validate each input field.
- Display error messages if input is invalid.
- Prevent form submission if validation fails.
- Display success when all inputs are valid.

**HTML (index.html)**

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Simple Form</title>

    <link rel="stylesheet" href="style.css">

    <script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.9.0/validator.min.js"></script>

</head>

<body>

<div class="container">

    <h2>User Details Form</h2>

    <form id="myForm" novalidate>

        <div class="form-group">

            <label>Name</label>

            <input type="text" id="name">

            <small class="hint">Only alphabets allowed</small>

            <span class="error" id="nameError"></span>

        </div>
```

```
        <div class="form-group">
            <label>Email</label>
            <input type="text" id="email">
            <small class="hint">Example: abc@gmail.com</small>
            <span class="error" id="emailError"></span>
        </div>
        <div class="form-group">
            <label>Phone Number</label>
            <input type="text" id="phone">
            <small class="hint">10 digits, starts with 6–9</small>
            <span class="error" id="phoneError"></span>
        </div>
        <button type="submit">Submit</button>
    </form>
</div>
<script src="script.js"></script>
</body>
</html>
```

CSS (style.css)

```
body {
    font-family: Arial, sans-serif;
    background: #f4f4f4;
    margin: 0;
```

```
}
.container {
    width: 340px;
    margin: 40px auto;
    background: white;
    padding: 20px 22px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
h2 {
    text-align: center;
    margin-bottom: 15px;
    color: #333;
}
.form-group {
    margin-bottom: 10px;
}
label {
    font-weight: 600;
    font-size: 13px;
    display: block;
    margin-bottom: 3px;
}
input {
```

```
    width: 100%;
    padding: 8px;
    border: 1px solid #bbb;
    border-radius: 5px;
    font-size: 13px;
}
input:focus {
    border-color: #007bff;
    outline: none;
}
.hint {
    font-size: 10.5px;
    color: #666;
    margin-top: 2px;
    display: block;
}
.error {
    font-size: 11.5px;
    color: #d9534f;
    margin-top: 2px;
    min-height: 12px;
    display: block;
}
input.error-border {
```

```
    border: 1px solid #d9534f;
    background: #fff7f7;
}
button {
    width: 100%;
    padding: 9px;
    background: #007bff;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 14px;
    margin-top: 8px;
    cursor: pointer;
}
button:hover {
    background: #0056b3;
}
```

**JavaScript (script.js)**

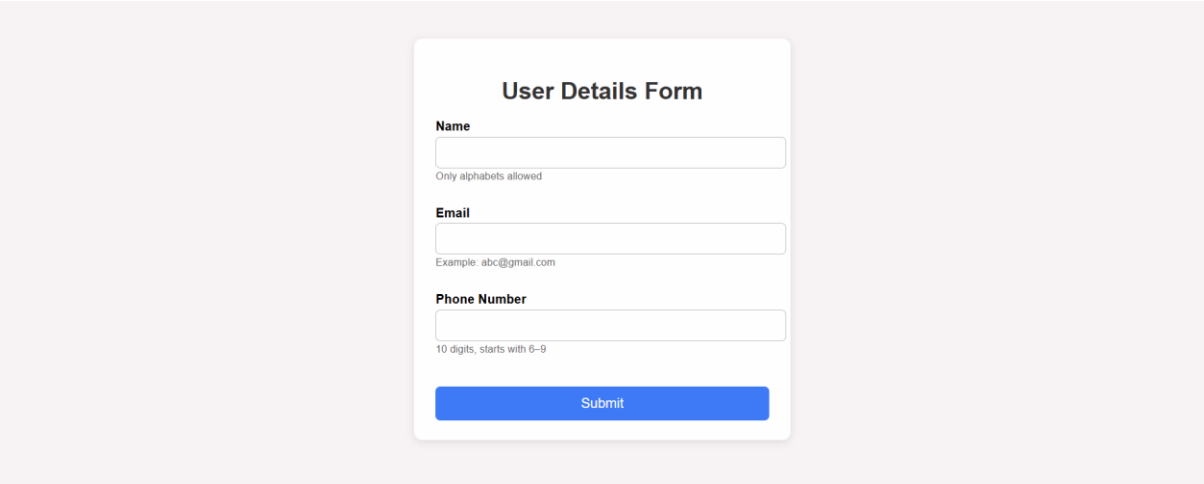
```
document.getElementById("myForm").addEventListener("submit", function(event) {
    const name = document.getElementById("name");
    const email = document.getElementById("email");
    const phone = document.getElementById("phone");
    let isValid = true;
```

```
document.querySelectorAll(".error").forEach(e => e.textContent = "");
document.querySelectorAll("input").forEach(i => i.classList.remove("error-border"));
if (validator.isEmpty(name.value.trim())) {
    setError(name, "nameError", "Name is required");
    isValid = false;
} else if (!validator.isAlpha(name.value.replace(/\s/g, "))) {
    setError(name, "nameError", "Only alphabets allowed");
    isValid = false;
}
if (validator.isEmpty(email.value.trim())) {
    setError(email, "emailError", "Email is required");
    isValid = false;
} else if (!validator.isEmail(email.value)) {
    setError(email, "emailError", "Enter valid email");
    isValid = false;
}
if (validator.isEmpty(phone.value.trim())) {
    setError(phone, "phoneError", "Phone number is required");
    isValid = false;
} else if (!/^[6-9]\d{9}$/.test(phone.value)) {
    setError(phone, "phoneError", "Enter valid 10-digit number");
    isValid = false;
}
if (!isValid) {
```

```
        event.preventDefault();
    }
});
function setError(element, id, message) {
    document.getElementById(id).textContent = message;
    element.classList.add("error-border");
}
```

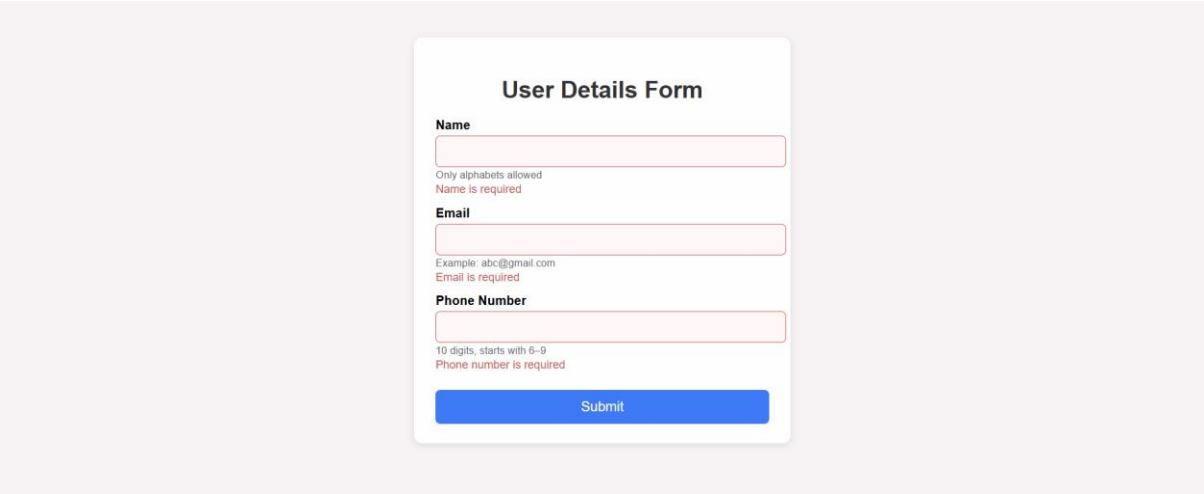
## SCREENS

### User Input Form Screen




This is the main screen of the application. It contains a user details form where the user can enter their details such as Name, Email, and Phone Number. The screen is designed using HTML and styled using CSS to provide a clean and user-friendly interface. Each input field has a label and a placeholder to guide the user. Below each input field, a hint is displayed to help the user understand the expected format of input. If the user enters invalid data, error messages are shown below the respective fields. The form ensures validation using JavaScript and Validator.js before submission. If any field contains incorrect data, the form will not be submitted until all errors are corrected.

### Validation Error Screen (All Fields Empty Case)



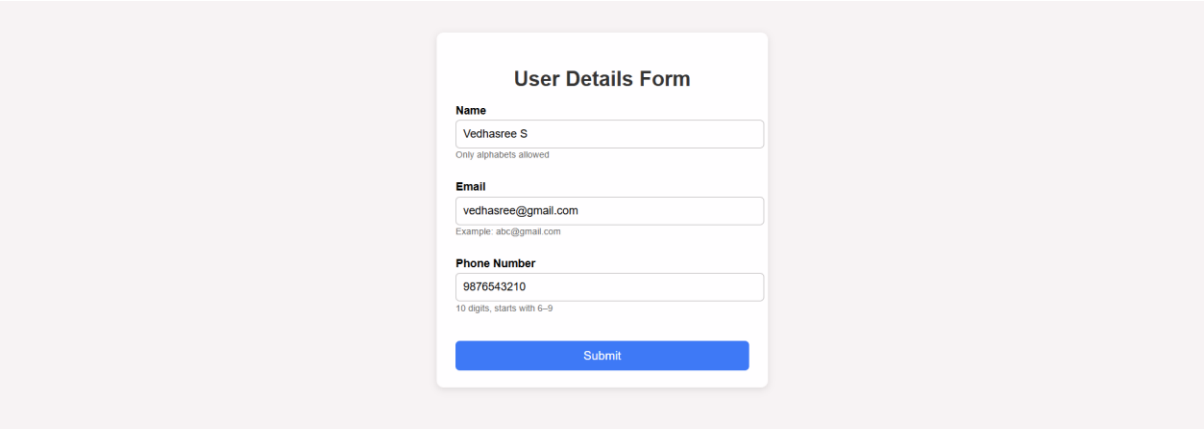

This screen appears when the user clicks the **Submit button without entering any data** in the form fields. The form remains visible, but error messages are displayed below each input field indicating that the required information is missing.

Since all fields are empty, validation is triggered for all inputs at once:

- Name field shows “Name is required”
- Email field shows “Email is required”
- Phone field shows “Phone number is required”

The input boxes are highlighted with error styling to clearly indicate invalid or missing data.

### Successful Submission Screen (All Inputs Valid)



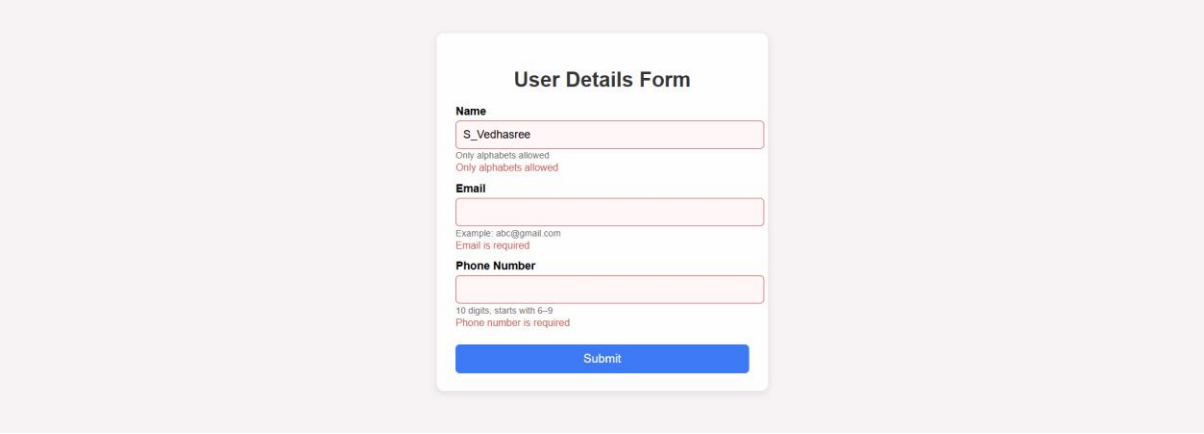
This screen appears when the user enters valid data in all input fields and clicks the Submit button.

All validations are successfully passed:

- Name contains only alphabets
- Email is in correct format
- Phone number is a valid 10-digit number starting with 6–9

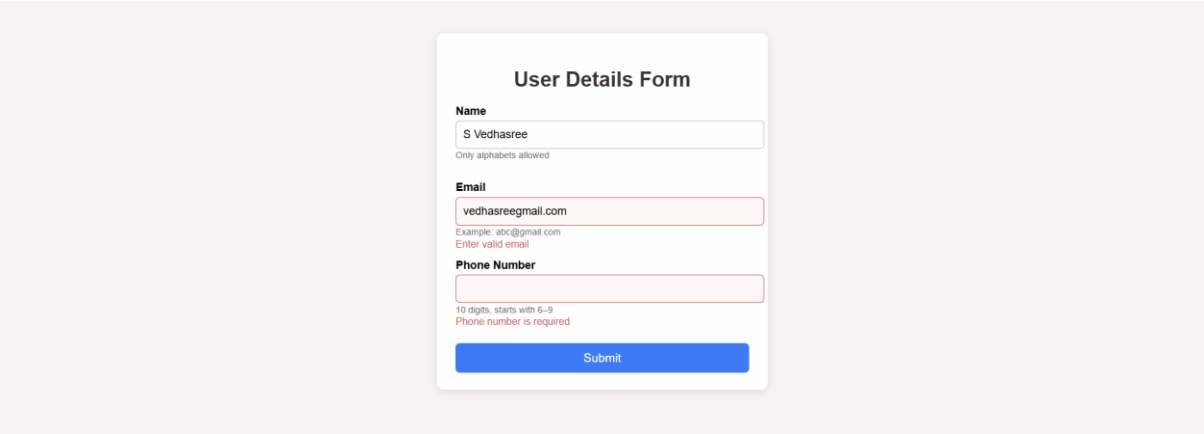
Since all inputs are valid, no error messages are displayed below any field. The form is successfully submitted (or allowed to proceed), indicating that the entered data meets all validation rules defined using JavaScript and Validator.js.

**Name Field Validation Error Screen (Invalid Name Case)**



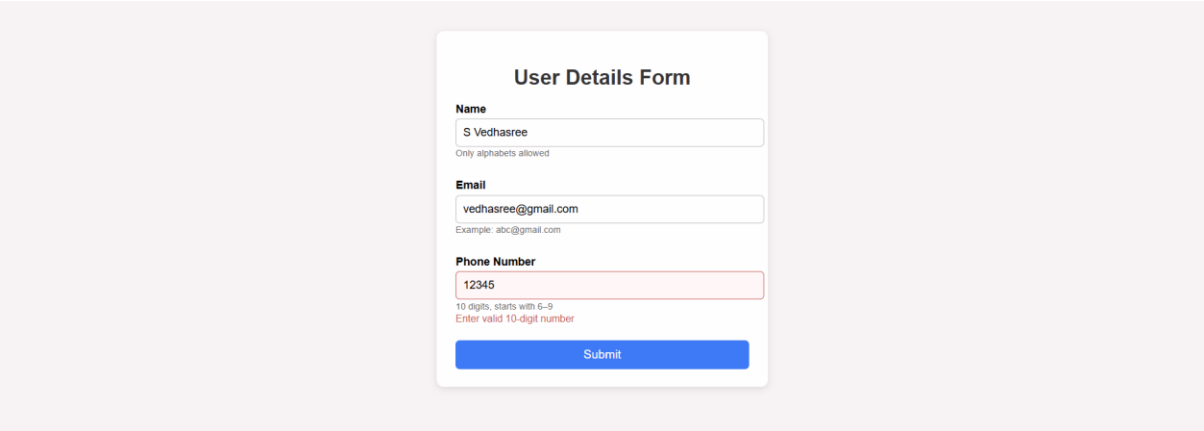

This screen appears when the user enters an invalid name containing characters other than alphabets, such as an underscore (_), numbers, or special characters, and clicks the Submit button. The validation logic checks that the name field should contain only alphabets and spaces. Since the entered name includes an invalid character (_), the system identifies it as incorrect input. As a result, an error message is displayed below the Name field indicating that only alphabets are allowed. The input box is also highlighted to show the validation error. This ensures that the user enters a properly formatted name and maintains data consistency.

**Email Validation Error Screen (Missing “@” Symbol Case)**




The validation logic checks whether the email follows a proper format (local-part + “@” + domain). Since the entered email does not contain “@”, it is considered invalid. As a result, an error message is displayed below the Email field indicating that the user must enter a valid email address. The input field is also highlighted to show the validation error. This ensures that only correctly formatted email addresses are accepted before form submission.

**Phone Number Validation Error Screen (Invalid Length Case)**




This screen appears when the user enters an incomplete or invalid phone number such as **“12345”** and clicks the Submit button. The validation logic checks that the phone number must be exactly **10 digits** and must start with digits **6–9** (as per Indian mobile number format). Since the entered value contains only 5 digits and does not meet the required format, it is considered invalid. As a result, an error message is displayed below the Phone Number field indicating that a valid 10-digit number must be entered. The input box is also highlighted to show the validation error. This ensures that only properly formatted mobile numbers are accepted before form submission.